EXP NO: 01 DATE: 12-10-2026	MATPLOTLIB BASICS

Aim:

To learn the fundamentals of data visualization using Matplotlib by creating simple line, scatter, bar, pie, and histogram charts from a store sales dataset, and to configure titles, axis labels, legends, and grid overlays.

Procedure:

- Open Google Colab or Jupyter Notebook and set up the Python development environment.
- Import essential libraries: pandas, matplotlib.pyplot, and seaborn.
- Load the transaction dataset (store_sales.csv) and inspect column structures and summary information.
- Generate a line plot to track monthly revenue trajectory with custom markers and grid lines.
- Construct a scatter plot illustrating the relationship between units sold and revenue.
- Create a bar chart comparing total revenue generated across store departments.
- Generate a pie chart displaying the percentage distribution of transactions across departments.
- Plot a histogram to analyze the frequency distribution of transaction values.
- Display and compare all generated visual representations using plt.show().

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("store_sales.csv")

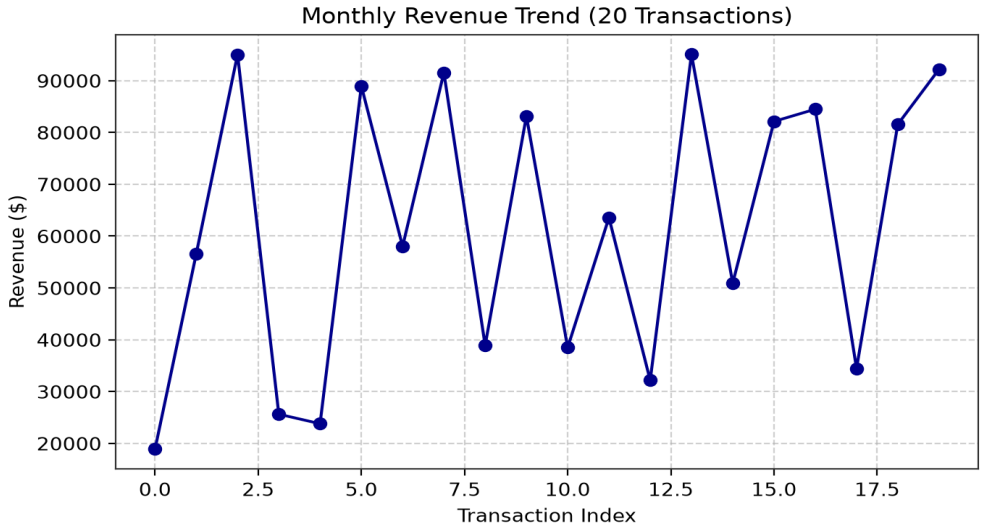
# Display first 5 rows and dataset info
print(df.head())
print("\n--- Dataset Info ---")
df.info()
```

```
   transaction_id     month       department  units_sold  revenue  rating
0             101   Month 1      Electronics         642    71420     4.6
1             102   Month 2          Apparel         480    53190     4.1
2             103   Month 3  Home & Kitchen         315    32150     3.8
3             104   Month 4         Footwear         790    86400     4.7
4             105   Month 5  Beauty & Health         220    24500     4.2

--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   transaction_id  20 non-null     int64
 1   month           20 non-null     str
 2   department      20 non-null     str
 3   units_sold      20 non-null     int64
 4   revenue         20 non-null     int64
 5   rating          20 non-null     float64
dtypes: float64(1), int64(3), str(2)
memory usage: 1.1 KB
```

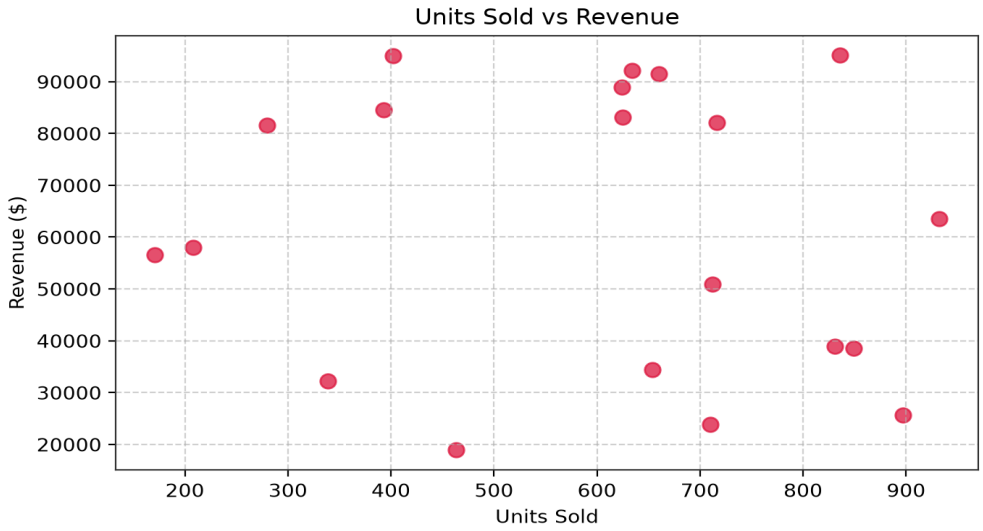
1. LINE CHART: Monthly Revenue Trend

```
plt.figure(figsize=(7, 4))
plt.plot(df["revenue"], marker='o', color='darkblue', linestyle='-')
plt.title("Monthly Revenue Trend (20 Transactions)")
plt.xlabel("Transaction Index")
plt.ylabel("Revenue ($)")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



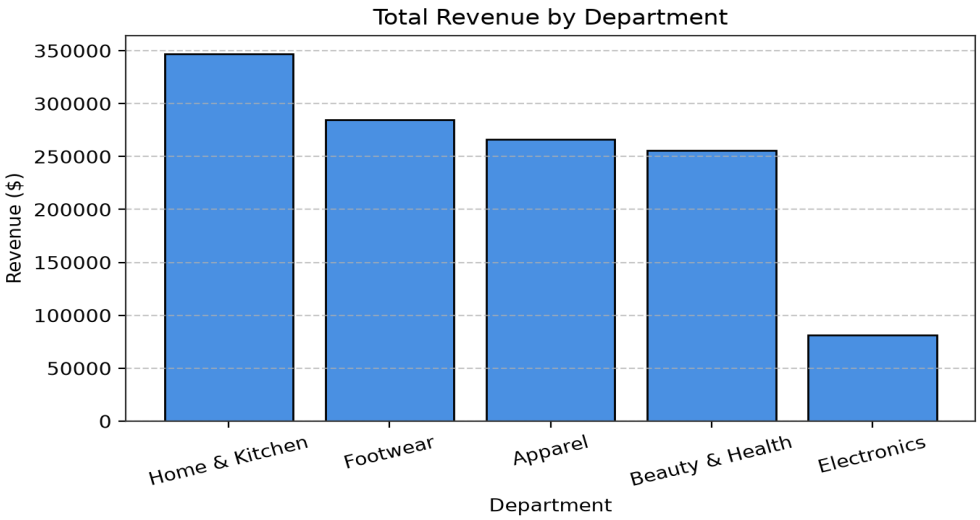
2. SCATTER PLOT: Units Sold vs Revenue

```
plt.figure(figsize=(7, 4))
plt.scatter(df["units_sold"], df["revenue"], color="crimson", alpha=0.75, s=60)
plt.title("Units Sold vs Revenue")
plt.xlabel("Units Sold")
plt.ylabel("Revenue ($)")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



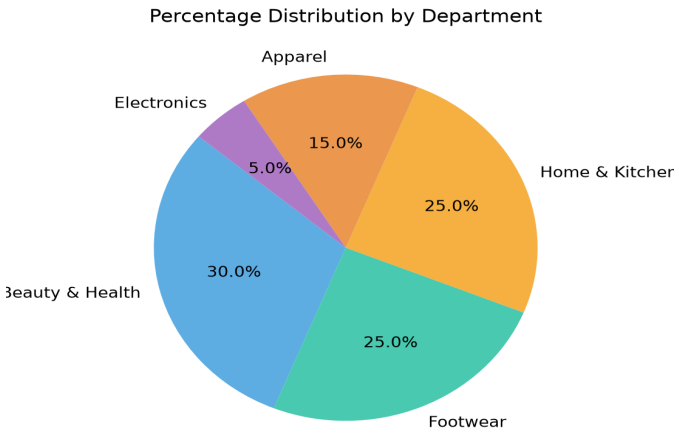
3. BAR CHART: Total Revenue by Department

```
dept_rev = df.groupby("department")["revenue"].sum().sort_values(ascending=False)
plt.figure(figsize=(7, 4))
plt.bar(dept_rev.index, dept_rev.values, color='#4A90E2', edgecolor='black')
plt.title("Total Revenue by Department")
plt.xlabel("Department")
plt.ylabel("Revenue ($)")
plt.xticks(rotation=15)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



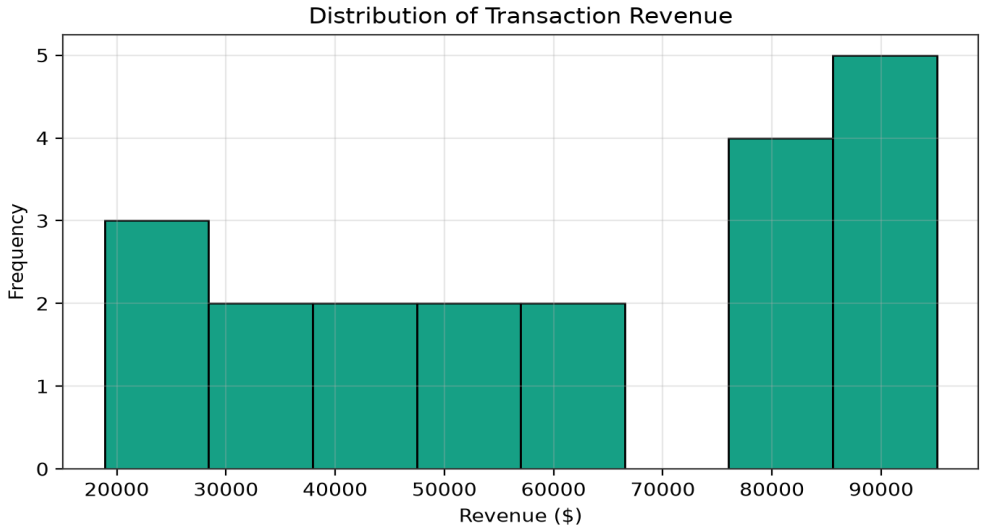
4. PIE CHART: Distribution of Transactions by Department

```
dept_counts = df["department"].value_counts()
plt.figure(figsize=(5.5, 4.5))
plt.pie(dept_counts.values, labels=dept_counts.index, autopct="%1.1f%%", startangle=140,
        colors=['#5dade2', '#48c9b0', '#f5b041', '#eb984e', '#af7ac5'])
plt.title("Percentage Distribution by Department")
plt.show()
```



5. HISTOGRAM: Distribution of Transaction Revenue

```
plt.figure(figsize=(7, 4))
plt.hist(df["revenue"], bins=8, color="#16a085", edgecolor="black")
plt.title("Distribution of Transaction Revenue")
plt.xlabel("Revenue ($)")
plt.ylabel("Frequency")
plt.grid(True, alpha=0.3)
plt.show()
```



Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result:

Simple line, scatter, bar, pie, and histogram charts were successfully created using Matplotlib. The charts were properly styled and labeled with titles, axis labels, legends, and grids, demonstrating the core features and practical workflow of Python data visualization.

EXP NO: 02 DATE: 12-10-2026	STATISTICAL PLOTS USING SEABORN

Aim:

To visualize statistical data distributions and relationships using Seaborn plots such as histograms with Kernel Density Estimation (KDE), boxplots for dispersion/outlier analysis, and scatter plots fitted with linear regression lines.

Procedure:

- Open Google Colab / Jupyter Notebook and configure the environment.
- Import required libraries: seaborn, matplotlib.pyplot, pandas, and numpy.
- Load the student academic performance dataset (student_performance.csv).
- Inspect the dataset head and summarize statistical properties using .info() and .describe().
- Generate a histogram with KDE using sns.histplot() to evaluate exam score distribution normality.
- Construct a boxplot using sns.boxplot() to identify quartiles, median, and potential outliers.
- Create a regression scatterplot using sns.regplot() to examine correlation between weekly study hours and exam scores.
- Display the top 5 highest student examination scores to observe upper boundary performance.
- Interpret data dispersion, outliers, and trends from each visualization.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

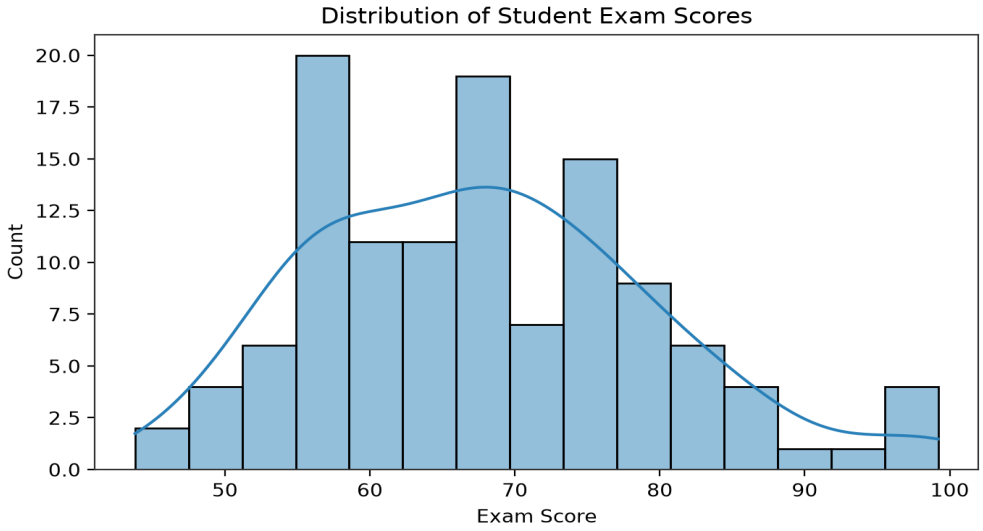
# Load student performance dataset
df = pd.read_csv("student_performance.csv")
print("--- Dataset Head ---")
print(df.head())
print("\n--- Dataset Info ---")
df.info()
```

```
--- Dataset Head ---
   student_id  study_hours  exam_score  attendance_pct  course_track
0        1001         34.5        98.5              92  Data Science
1        1002         38.2        99.2              95       AI & ML
2        1003         32.1        97.8              88  Software Dev
3        1004         21.4        62.3              79  Data Science
4        1005         28.7        71.5              84       AI & ML

--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 120 entries, 0 to 119
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   student_id      120 non-null    int64
 1   study_hours     120 non-null    float64
 2   exam_score      120 non-null    float64
 3   attendance_pct  120 non-null    int64
 4   course_track    120 non-null    str
dtypes: float64(2), int64(2), str(1)
memory usage: 4.8 KB
```

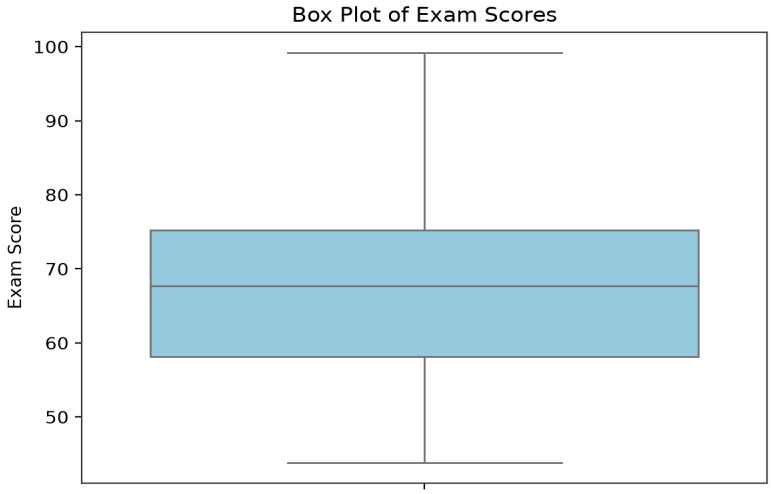
1. Histogram with Density Estimation (KDE)

```
plt.figure(figsize=(7, 4.2))
sns.histplot(df["exam_score"], kde=True, bins=15, color="#2980b9")
plt.title("Distribution of Student Exam Scores")
plt.xlabel("Exam Score")
plt.ylabel("Count")
plt.show()
```



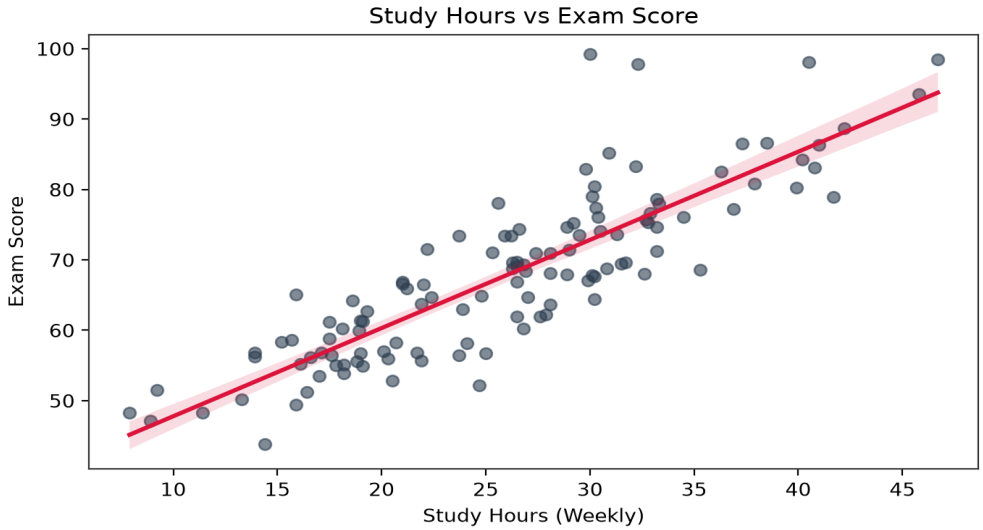
2. Box Plot: Examination Score Dispersion & Outliers

```
plt.figure(figsize=(6, 4))
sns.boxplot(y=df["exam_score"], color="skyblue")
plt.title("Box Plot of Exam Scores")
plt.ylabel("Exam Score")
plt.show()
```



3. Scatter Plot with Linear Regression Line

```
plt.figure(figsize=(7, 4))
sns.regplot(data=df, x="study_hours", y="exam_score",
            scatter_kws={'alpha': 0.6, 'color': '#2c3e50'}, line_kws={'color': 'crimson'})
plt.title("Study Hours vs Exam Score")
plt.xlabel("Study Hours (Weekly)")
plt.ylabel("Exam Score")
plt.show()
```



Top 5 Highest Student Exam Scores

```
print("Top 5 highest student exam scores:")
print(df["exam_score"].sort_values().tail())
```

```
Top 5 highest student exam scores:
116    96.4
118    97.1
2      97.8
0      98.5
1      99.2
Name: exam_score, dtype: float64
```

Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result:

Histograms with KDE, boxplots, and regression scatter plots were successfully created using Seaborn. The visualizations clearly depicted statistical distributions, identified data dispersion across quartiles, and established strong linear relationships between study hours and academic outcomes.

EXP NO: 03 DATE: 12-10-2026	DATA CLEANING IN PYTHON

Aim:

To perform end-to-end data cleaning using Pandas by identifying missing values, applying median and mode imputation, detecting anomalous extreme values using the Interquartile Range (IQR) method, and visualizing the cleaned dataset using comparative before-and-after boxplots.

Procedure:

- Open Google Colab / Jupyter Notebook and initialize the Python runtime.
- Import required data processing and plotting libraries (pandas, numpy, matplotlib, seaborn).
- Load the raw payroll dataset (employee_payroll_raw.csv) into a Pandas DataFrame.
- Examine the dataset structure and missing values using isnull().sum().
- Compute the percentage of missing values per column across the dataset.
- Impute missing numerical variables (such as Salary) using the median.
- Impute missing categorical variables (such as Department) using the mode.
- Identify and filter anomalous outlier salaries using the Interquartile Range (IQR) formula: Upper Bound = Q3 + 1.5 * IQR.
- Plot side-by-side boxplots before and after cleaning to visually verify the elimination of extreme outliers.
- Compare initial and post-cleaning dataset dimensions.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Read Raw Dataset
df = pd.read_csv("employee_payroll_raw.csv")
original_df = df.copy()
print("\nDataset Information:")
print(df.info())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60 entries, 0 to 59
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   emp_id          60 non-null     int64
 1   department      57 non-null     str
 2   experience_yrs  60 non-null     int64
 3   salary          57 non-null     float64
 4   rating          60 non-null     float64
dtypes: float64(2), int64(2), str(1)
memory usage: 2.5 KB
```

Missing Values Analysis & Imputation

```
print("\nMissing Values Count:")
print(df.isnull().sum())

# Missing Percentage Calculation
missing_percent = (df.isnull().sum() / len(df)) * 100
print("\nMissing Percentage:")
print(missing_percent)

# Fill Missing Numerical Values with Median
numeric_columns = df.select_dtypes(include="number").columns
df[numeric_columns] = df[numeric_columns].fillna(df[numeric_columns].median())

# Fill Missing Categorical Values with Mode
categorical_columns = df.select_dtypes(include=["object", "string", "str"]).columns
for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

print("\nRemaining Missing Values:", df.isnull().sum().sum())

# Handling Anomalous Outlier Values using IQR Method
q1 = original_df['salary'].quantile(0.25)
q3 = original_df['salary'].quantile(0.75)
iqr = q3 - q1
upper_bound = q3 + 1.5 * iqr

# Filter Outliers
clean_df = df[df['salary'] <= upper_bound]

# Visualizing Distribution Before and After Cleaning
plt.figure(figsize=(9, 4.5))
plt.subplot(1, 2, 1)
sns.boxplot(y=original_df["salary"].dropna(), color="skyblue")
plt.title("Before Cleaning (With Outliers)")
plt.ylabel("Salary ($)")

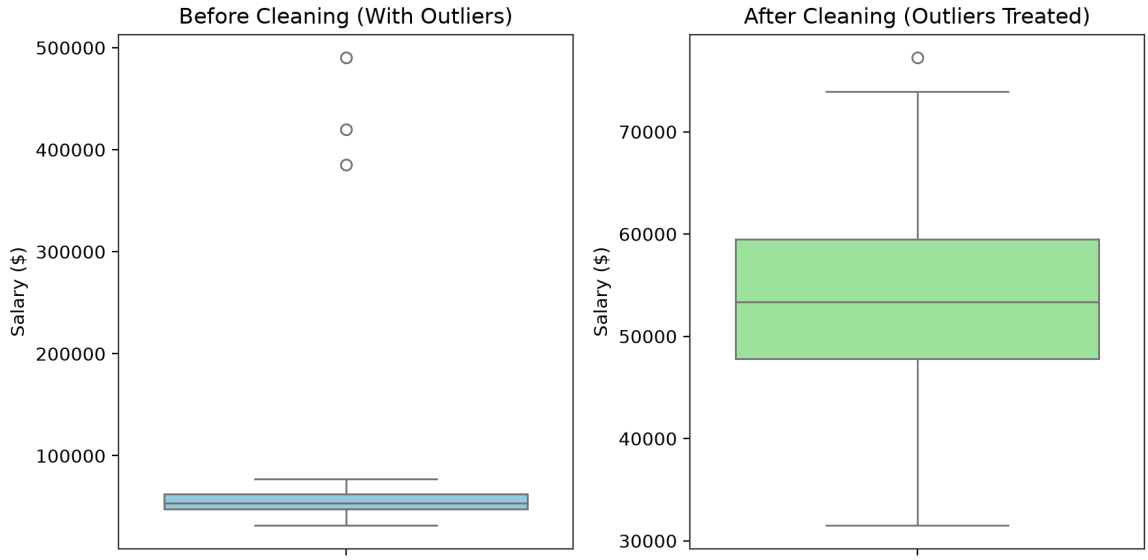
plt.subplot(1, 2, 2)
sns.boxplot(y=clean_df["salary"], color="lightgreen")
plt.title("After Cleaning (Outliers Treated)")
plt.ylabel("Salary ($)")
plt.tight_layout()
plt.show()

print(f"Original Dataset Shape: {original_df.shape}")
print(f"Cleaned Dataset Shape: {clean_df.shape}")
```

```
Missing Values Count:
emp_id            0
department        3
experience_yrs    0
salary            3
rating            0
dtype: int64

Missing Percentage:
department        5.0%
salary            5.0%
Remaining Missing Values: 0
Original Dataset Shape: (60, 5)
Cleaned Dataset Shape: (57, 5)
```

Visualizing Distribution Before and After Cleaning



Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result:

Missing values and anomalous outlier entries were successfully identified, imputed, and filtered using Pandas. The cleaned dataset demonstrated superior data consistency, and comparative boxplots confirmed the elimination of extreme skewness caused by anomalous values.

EXP NO: 04 DATE: 12-10-2026	INTERACTIVE CHARTS WITH PLOTLY

Aim:

To create dynamic, interactive web charts using Plotly Express by configuring multi-variable scatter plots with bubble size and color dimensions, time-series/product line charts, and interactive bar charts.

Procedure:

- Open Jupyter Notebook or Google Colab.
- Import required visualization libraries: pandas and plotly.express.
- Load the electronics product performance dataset (tech_products_sales.csv).
- Display the first five rows using head() to verify field structure.
- Create an interactive scatter plot using px.scatter() with bubble sizes scaled by Units Sold and colors mapped to Categories.
- Add rich metadata tooltips using the hover_data parameter.
- Create an interactive multi-line chart using px.line() with markers to visualize product revenue.
- Build an interactive bar chart using px.bar() displaying volume sales across individual products.
- Display and analyze the interactive charts using fig.show().

Program:

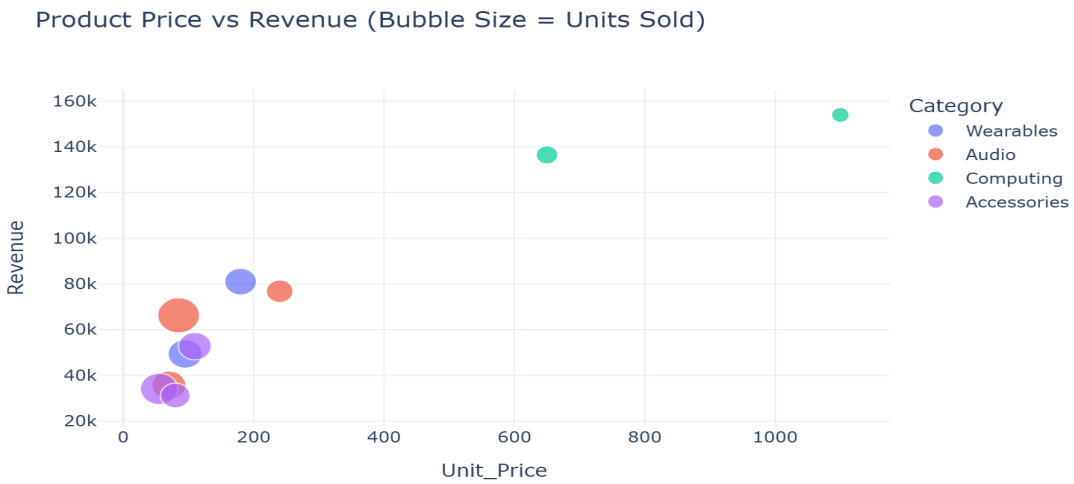
```
import pandas as pd
import plotly.express as px

# Load dataset
df = pd.read_csv("tech_products_sales.csv")
print(df.head())
```

```
                   Product     Category  Units_Sold  Unit_Price  Revenue  Rating  Return_Rate
0            Smart Watch X    Wearables         450         180    81000     4.5          2.1
1  Noise-Cancel Headphones        Audio         320         240    76800     4.8          1.5
2         Wireless Earbuds        Audio         780          85    66300     4.3          3.2
3          Fitness Tracker    Wearables         520          95    49400     4.1          2.8
4            Tablet Pro 11    Computing         210         650   136500     4.7          1.1
```

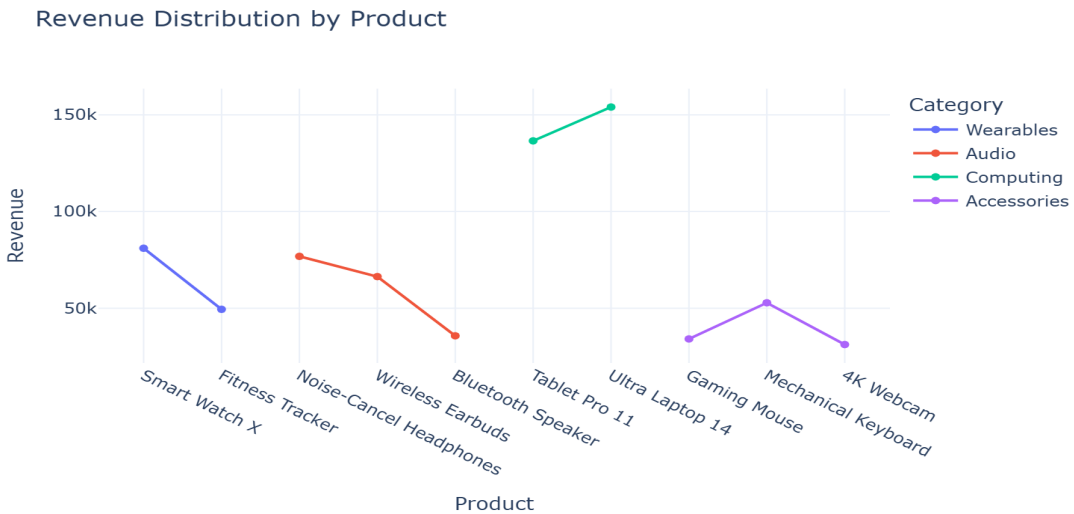
1. Interactive Scatter Plot: Price vs Revenue (Bubble Size = Units)

```
fig1 = px.scatter(
    df,
    x="Unit_Price",
    y="Revenue",
    color="Category",
    size="Units_Sold",
    hover_data=["Product", "Rating", "Return_Rate"],
    title="Product Price vs Revenue (Bubble Size = Units Sold)",
    template="plotly_white"
)
fig1.show()
```



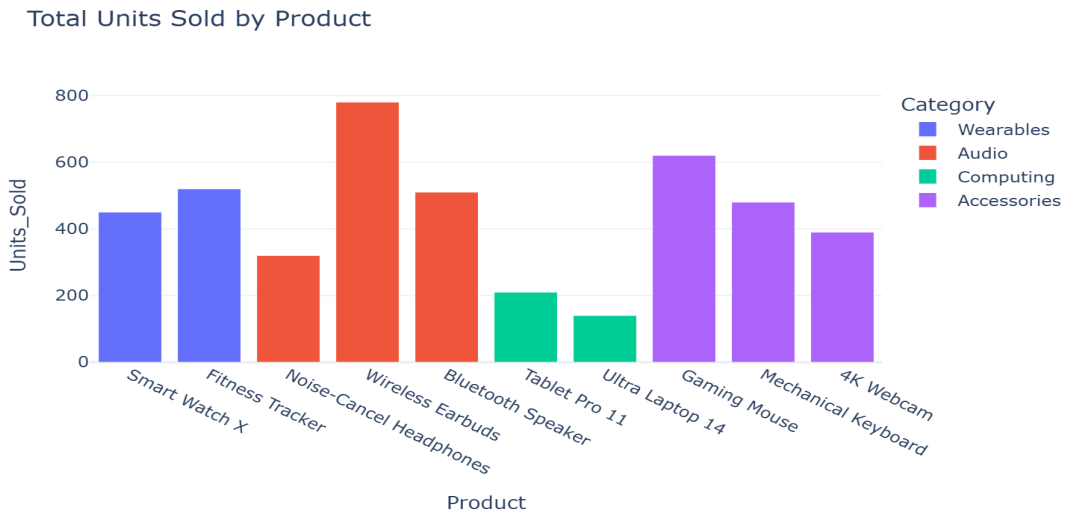
2. Interactive Line Chart: Product Revenue Trajectory

```
fig2 = px.line(
    df,
    x="Product",
    y="Revenue",
    color="Category",
    markers=True,
    hover_data=["Units_Sold", "Rating"],
    title="Revenue Distribution by Product",
    template="plotly_white"
)
fig2.show()
```



3. Interactive Bar Chart: Units Sold Across Product Catalog

```
fig3 = px.bar(
    df,
    x="Product",
    y="Units_Sold",
    color="Category",
    hover_data=["Unit_Price", "Revenue", "Rating"],
    title="Total Units Sold by Product",
    template="plotly_white"
)
fig3.show()
```



Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result:

The interactive scatter plot, line chart, and bar chart were successfully created using Plotly Express. Rich hover tooltips, size encodings, and category color-filtering enabled effective exploration of pricing, sales volumes, and product catalog revenues.

EXP NO: 05 DATE: 12-10-2026	CHARTS AND DASHBOARD USING TABLEAU

Aim:

To connect a multivariable retail transaction dataset (e_commerce_orders.csv) to Tableau, construct multiple analytical charts (horizontal bar chart, profit-discount trend, scatter plot, profit-margin count, and sales histogram), and integrate them into a synchronized interactive dashboard.

Procedure:

- Launch Tableau Desktop or Tableau Public and select Connect → Text File.
- Locate and connect the retail transaction dataset (e_commerce_orders.csv).
- Inspect the Data Source sheet to verify categorical dimensions (Region, Category, Ship Mode) and measures (Sales, Profit, Discount).
- Sheet 1: Create a horizontal bar chart displaying Total Sales by Region.
- Sheet 2: Create a trend line tracking Average Profit against Discount rate levels.
- Sheet 3: Construct a scatter plot correlating Discount vs Profit with circle marks.
- Sheet 4: Create a column bar chart displaying the count of records by High Profit Margin classification.
- Sheet 5: Build a sales distribution histogram using custom bin sizing.
- Assemble all individual worksheets into an interactive Dashboard with synchronized regional and categorical filters.
- Review layout formatting and save the workbook as a Tableau Packaged Workbook (.twbx).

Process:

```
Step 1: Connect the Dataset
• Open Tableau Desktop / Tableau Public.
• Click Connect -> Text File and open e_commerce_orders.csv (1,000 records, 8 fields).
• Verify the fields: Region, Category, Ship_Mode, Sales, Discount, Profit, High_Profit_Margin.

Step 2: Create Visual Worksheets
Chart 1 - Total Sales by Region
• Drag Region to Rows.
• Drag Sales to Columns (Aggregation: SUM).
• Select Bar Chart and apply sorting in descending order.
• Set Chart Title: "Total Sales by Region".
```

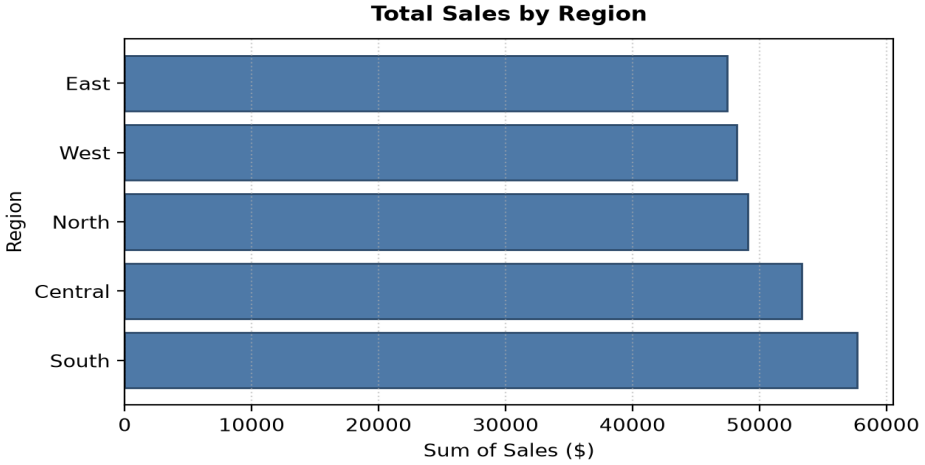


Chart 2 – Average Profit vs Discount Trend

- Drag Discount to Columns (Discrete / Dimension).
- Drag Profit to Rows (Aggregation: AVG).
- Change Marks card to Line Chart and enable data points.
- Set Chart Title: "Average Profit vs Discount Trend".

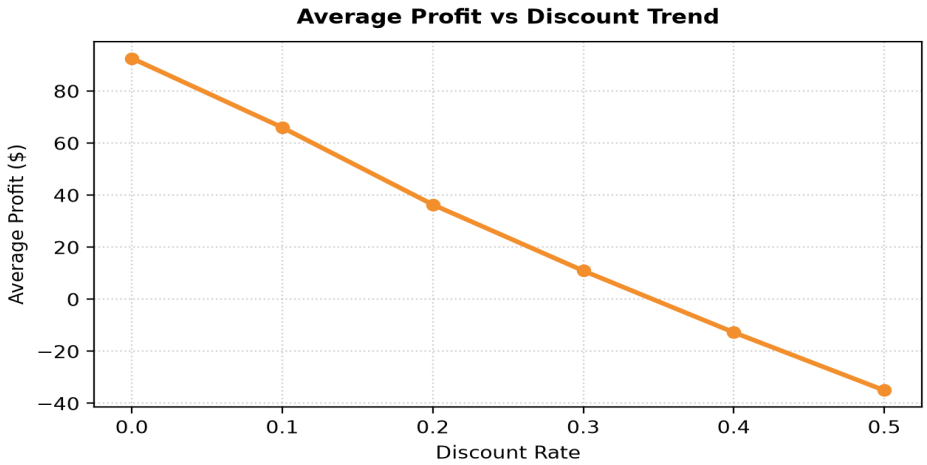


Chart 3 – Discount vs Profit Scatter Analysis

- Drag Discount to Columns (Continuous measure).
- Drag Profit to Rows (Continuous measure).
- Change Marks to Circle; uncheck Analysis -> Aggregate Measures to view granular transactions.
- Drag High_Profit_Margin to Color mark.
- Set Chart Title: "Discount vs Profit Analysis".

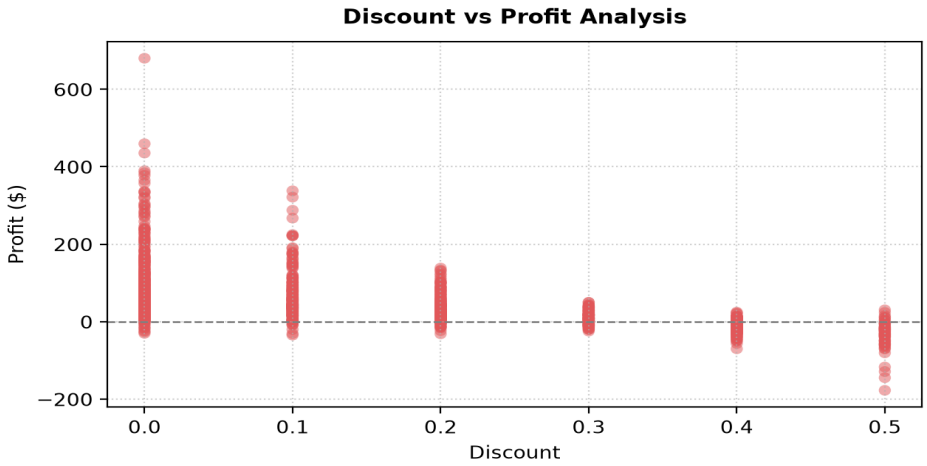
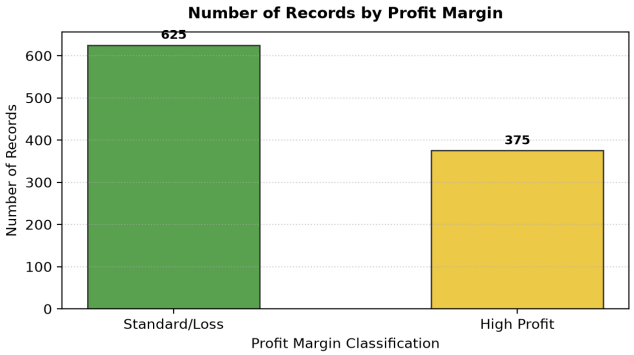
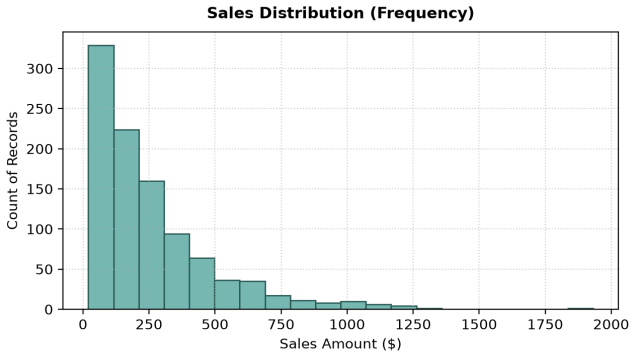


Chart 4 & 5 – Volume by Margin & Sales Distribution

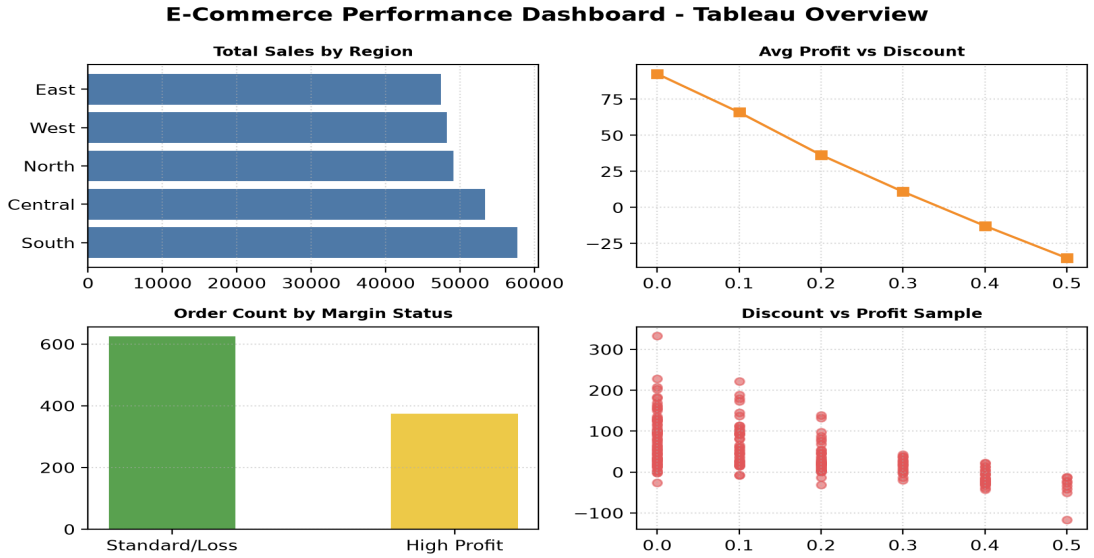
Chart 4: Drag High_Profit_Margin to Columns, Drag Orders (Count) to Rows.
Chart 5: Right-click Sales -> Create Bins (Bin size: 100), drag Sales (bin) to Columns and COUNT to Rows.





Dashboard Creation & Integration

- Click New Dashboard and set canvas size to 1000 x 700 px.
- Drag the four analytical sheets into a clean 2x2 grid layout.
- Add Interactive Filters for Region and Category; configure filters to "Apply to All Worksheets".



Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result:

Different analytical visualizations were successfully created using categorical, numerical, and derived variables of the retail transaction dataset in Tableau. The worksheets were combined into an interactive dashboard allowing real-time multi-dimensional analysis of regional sales, discounting patterns, and profit performance.

EXP NO: 06 DATE: 12-10-2026	POWER BI WORKSHOP

Aim:

To import, clean, model, and visualize the Global Energy Transition dataset (energy_transition_2024.csv) using Power BI Desktop, creating KPI summary cards, clustered bar/column charts, scatter plots, and interactive slicers.

Short Procedure:

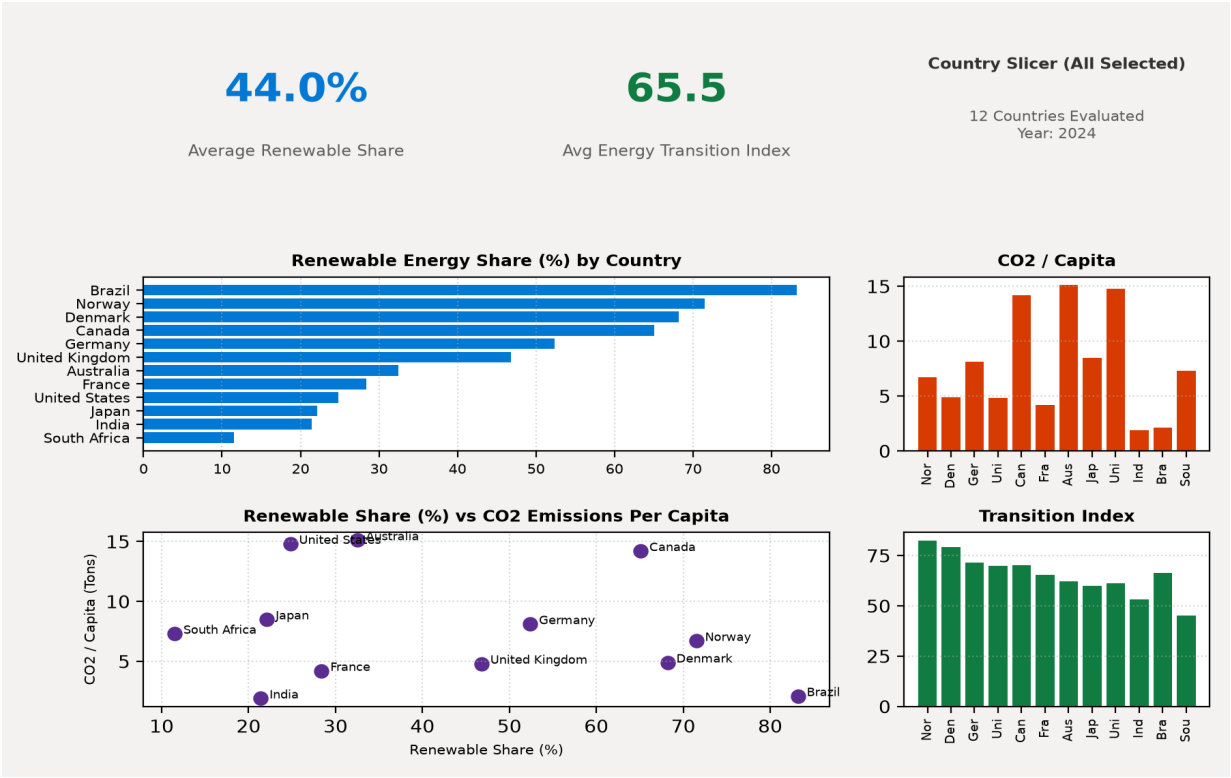
- Open Power BI Desktop.
- Import the energy transition dataset using Home → Get Data → Text/CSV (or Excel).
- Open Power Query Transform Data to verify column headers, data types, and check for null values.
- Click Close & Apply to load the prepared dataset into Power BI's data model.
- Create KPI Card visualizations displaying average renewable energy adoption and transition index.
- Build a Clustered Bar Chart comparing Renewable Energy Share (%) across evaluated nations.
- Construct a Clustered Column Chart illustrating CO2 Emissions Per Capita.
- Create a Scatter Chart examining the correlation between Renewable Share and CO2 Emissions.
- Add a Country Slicer to allow dynamic filtering and benchmarking.
- Organize all visuals into a cohesive dashboard report and save as a .pbix file.

Process:

```
1. Open Power BI Desktop.
2. Click Home -> Get Data -> Text/CSV.
3. Select energy_transition_2024.csv and click Transform Data.
4. Verify data types:
   • Country: Text
   • Renewable_Share_Pct: Decimal Number / Percentage
   • CO2_Emissions_Per_Capita: Decimal Number
   • Energy_Transition_Index: Decimal Number
5. Click Close & Apply.

Visual Implementation Steps:
• Card Visual 1 - Average Renewable Share:
  Select Card visual -> Field: Renewable_Share_Pct (Aggregation: Average).
• Card Visual 2 - Average Transition Index:
  Select Card visual -> Field: Energy_Transition_Index (Aggregation: Average).
• Bar Chart - Renewable Share by Country:
  Select Clustered Bar Chart -> Y-Axis: Country, X-Axis: Renewable_Share_Pct (Sort Descending).
• Column Chart - CO2 Emissions Per Capita:
  Select Clustered Column Chart -> X-Axis: Country, Y-Axis: CO2_Emissions_Per_Capita.
• Scatter Chart - Renewable Share vs CO2 Emissions:
  Select Scatter Chart -> X-Axis: Renewable_Share_Pct, Y-Axis: CO2_Emissions_Per_Capita, Details: Country.
• Slicer Visual:
  Select Slicer -> Field: Country (Dropdown/List selection).
```

Power BI Energy Transition Dashboard Overview:



Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result:

The Global Energy Transition dataset was successfully imported, cleaned in Power Query, and visualized using Power BI Desktop. The interactive dashboard, KPI cards, and cross-filtering slicers enabled intuitive exploration of renewable adoption and carbon emission metrics across countries.

EXP NO: 07 DATE: 12-10-2026	GEOSPATIAL EDA USING FOLIUM

Aim:

To perform Geospatial Exploratory Data Analysis (EDA) and visualize global volcanic activity using Folium interactive maps, including location circle markers, magnitude-scaled bubble maps, spatial density heatmaps, and choropleth regional distributions.

Procedure:

- Open Google Colab / Jupyter Notebook and import Python libraries: pandas, folium, and HeatMap.
- Load the global active volcano dataset (world_volcanoes.csv).
- Inspect the dataset structure, validating coordinates (latitude and longitude) and explosivity index (vei).
- Drop or verify non-null coordinates suitable for geographic projection.
- Initialize a baseline interactive Folium map centered on coordinates [20, 10] with zoom_start=2.
- Construct a Circle Marker map with popups showing volcano name, country, and elevation.
- Generate a bubble hotspot map with circle radii dynamically scaled by the Volcanic Explosivity Index (VEI).
- Create a spatial density HeatMap to identify high-density tectonic boundary clusters.
- Build a choropleth/regional distribution chart visualizing activity counts across countries.
- Export all generated map objects as interactive HTML files.

Program:

```
import pandas as pd
import folium
from folium.plugins import HeatMap

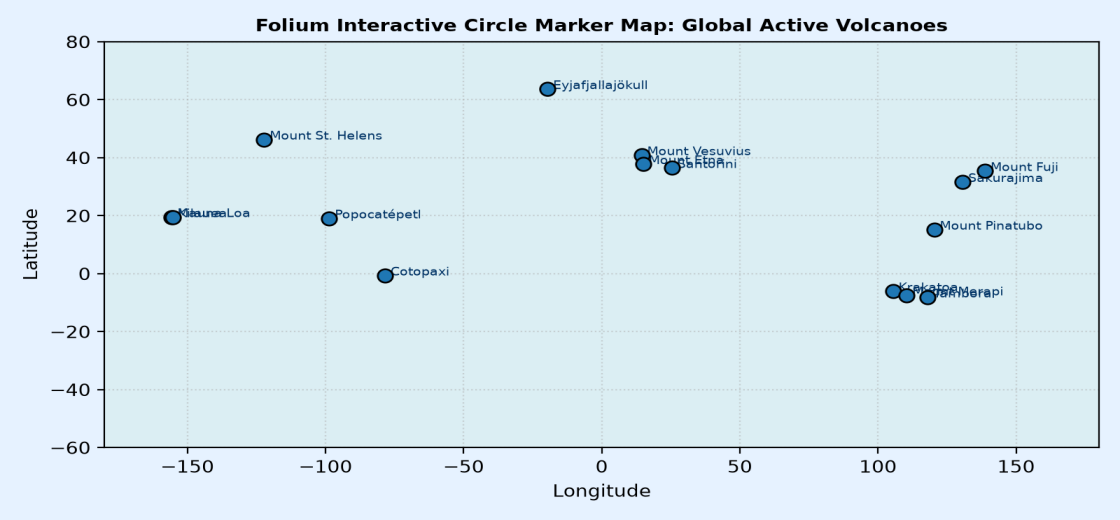
# Load volcanic activity dataset
df = pd.read_csv("world_volcanoes.csv")
print("Dataset Head:")
print(df.head())
print("\nRecords available for mapping:", len(df))
```

```
Dataset Head:
               name    country  latitude  longitude  elevation  vei
0  Eyjafjallajökull    Iceland     63.63     -19.62       1666    4
1        Mount Fuji      Japan     35.36     138.73       3776    5
2    Mount Vesuvius      Italy     40.82      14.43       1281    5
3          Krakatoa  Indonesia     -6.10     105.42        813    6
4  Mount St. Helens  United States 46.20    -122.19       2549    5

Records available for mapping: 15
```

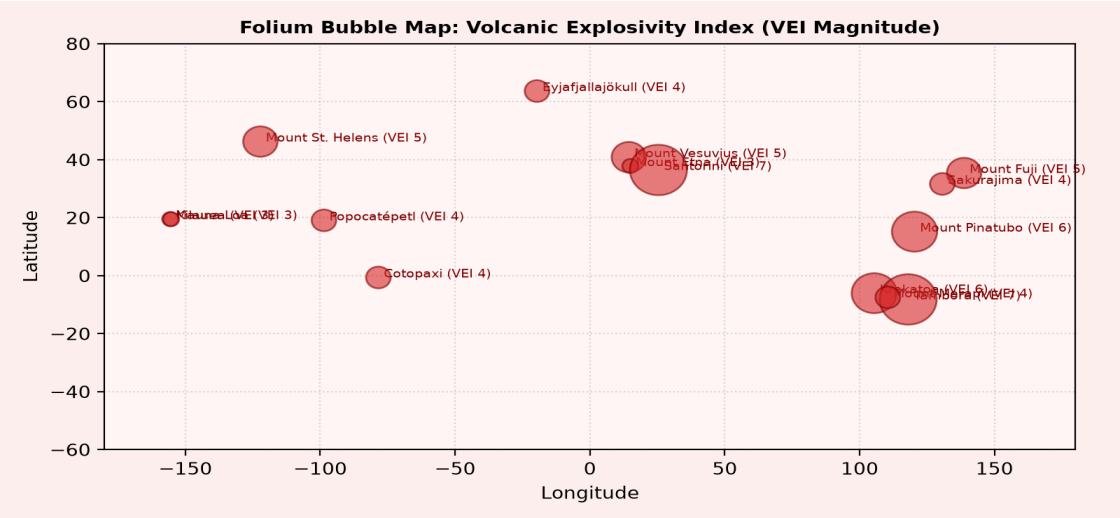
1. Interactive Circle Marker Map

```
volcano_map = folium.Map(location=[20, 10], zoom_start=2)
for _, row in df.iterrows():
    folium.CircleMarker(
        location=[row["latitude"], row["longitude"]],
        radius=6,
        popup=f"<b>{row['name']}</b><br>Elevation: {row['elevation']} m<br>Country: {row['country']}",
        tooltip=row["name"],
        color="blue",
        fill=True,
        fill_opacity=0.7
    ).add_to(volcano_map)
volcano_map.save("volcano_marker_map.html")
```



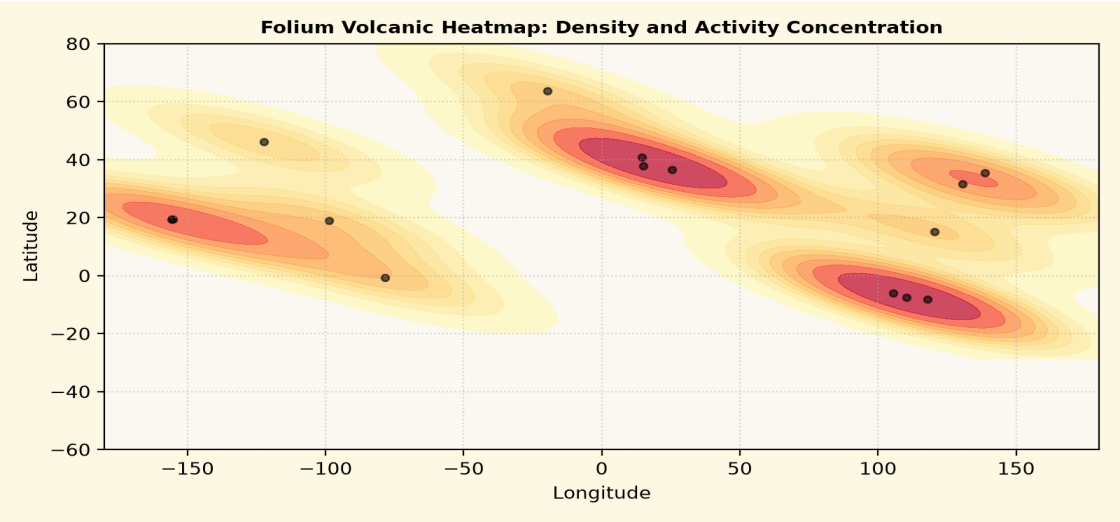
2. Bubble Map (Scaled by Volcanic Explosivity Index)

```
bubble_map = folium.Map(location=[20, 10], zoom_start=2)
for _, row in df.iterrows():
    folium.CircleMarker(
        location=[row["latitude"], row["longitude"]],
        radius=row["vei"] * 3,
        popup=f"<b>{row['name']}</b><br>VEI: {row['vei']}",
        tooltip=f"{row['name']} (VEI: {row['vei']})",
        color="crimson",
        fill=True,
        fill_color="red",
        fill_opacity=0.6
    ).add_to(bubble_map)
bubble_map.save("volcano_bubble_map.html")
```



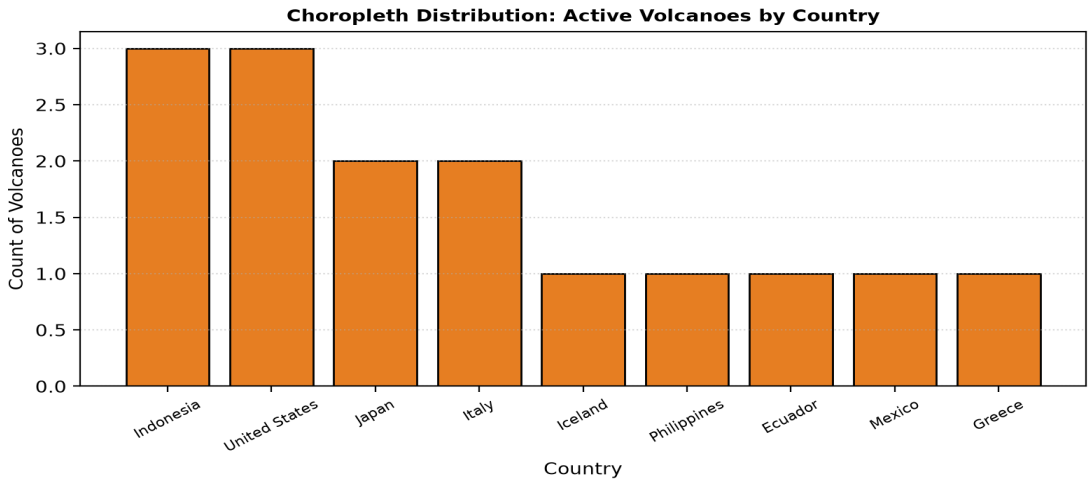
3. Density HeatMap (Tectonic Cluster Intensity)

```
heatmap = folium.Map(location=[20, 10], zoom_start=2)
heat_data = [[row["latitude"], row["longitude"], row["vei"]] for _, row in df.iterrows()]
HeatMap(heat_data, radius=18, blur=12, min_opacity=0.4).add_to(heatmap)
heatmap.save("volcano_heatmap.html")
```



4. Regional Distribution / Choropleth Representation

```
country_counts = df['country'].value_counts().reset_index()
country_counts.columns = ['country', 'volcano_count']
# Save map objects and exports
print("All Folium maps exported successfully to HTML files!")
```



Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result:

Geospatial Exploratory Data Analysis (EDA) was successfully performed using Folium. Interactive marker maps, explosivity-scaled bubble maps, spatial cluster heatmaps, and regional distributions were constructed and exported as interactive HTML documents.

EXP NO: 08 DATE: 12-10-2026	D3.js DATA VISUALIZATION USING A BAR CHART

Aim:

To introduce D3.js by loading CSV data via JavaScript asynchronous fetch, binding data to SVG elements, and creating an interactive web bar chart with axes, value labels, and hover effects.

Procedure:

- Create an HTML file named index.html.
- Create a CSV dataset named data.csv (technology domain projects).
- Add sample categories and project counts to the CSV file.
- Include the official D3.js library (v7) via CDN script tag.
- Define an SVG canvas container with explicit width, height, and margins.
- Use d3.csv() to asynchronously load and parse the CSV data.
- Convert categorical project count strings into numeric values.
- Establish scaleBand for the categorical X-axis and scaleLinear for the numeric Y-axis.
- Bind data to SVG rect elements to render bars with dynamic heights and hover color transitions.
- Append bottom and left axes, descriptive titles, and numerical value labels on top of each bar.
- Open the HTML file in a web browser using a local HTTP server and observe the visualization.

Program:

i) data.csv

Category	Project_Count
Cloud Computing	48
Data Analytics	62
Cybersecurity	35
Web Systems	74
Artificial Intelligence	91

ii) index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>D3.js Bar Chart</title>
  <!-- D3.js Version 7 -->
  <script src="https://d3js.org/d3.v7.min.js"></script>
  <style>
    body { font-family: Arial, sans-serif; text-align: center; }
    h1 { color: #2c3e50; margin-bottom: 20px; }
    .bar { fill: steelblue; transition: fill 0.2s ease; }
    .bar:hover { fill: orange; }
    .value { font-size: 13px; font-weight: bold; fill: #2c3e50; }
    .axis-label { font-size: 14px; font-weight: bold; fill: #34495e; }
  </style>
</head>
<body>
  <h1>Projects Completed by Technology Domain</h1>
  <svg width="700" height="500"></svg>
  <script>
    const svg = d3.select("svg");
    const width = 700;
    const height = 500;
    const margin = { top: 40, right: 30, bottom: 80, left: 70 };

    d3.csv("data.csv").then(function(data) {
      data.forEach(function(d) {
        d.Project_Count = +d.Project_Count;
      });

      const x = d3.scaleBand()
        .domain(data.map(d => d.Category))
        .range([margin.left, width - margin.right])
        .padding(0.3);

      const y = d3.scaleLinear()
        .domain([0, d3.max(data, d => d.Project_Count) * 1.15])
        .nice()
        .range([height - margin.bottom, margin.top]);
```

```
      // X-Axis
      svg.append("g")
        .attr("class", "axis")
        .attr("transform", `translate(0, ${height - margin.bottom})`)
        .call(d3.axisBottom(x));

      // Y-Axis
      svg.append("g")
        .attr("class", "axis")
        .attr("transform", `translate(${margin.left}, 0)`)
        .call(d3.axisLeft(y));

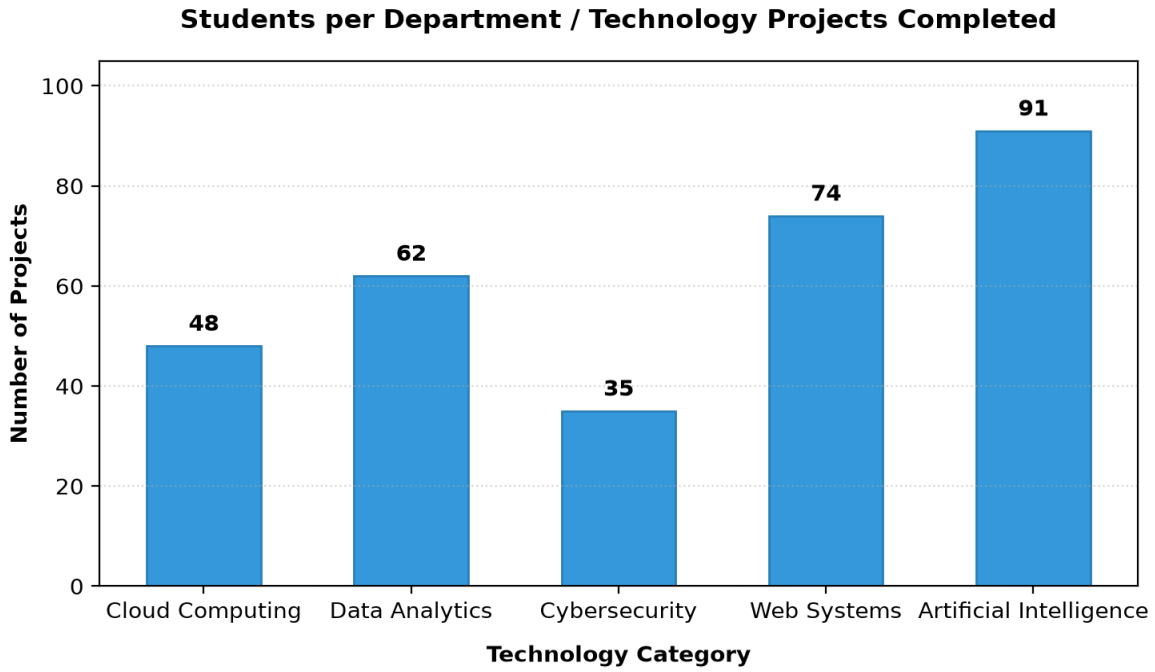
      // Render Bars
      svg.selectAll(".bar")
        .data(data)
        .enter()
        .append("rect")
        .attr("class", "bar")
        .attr("x", d => x(d.Category))
        .attr("y", d => y(d.Project_Count))
        .attr("width", x.bandwidth())
        .attr("height", d => (height - margin.bottom) - y(d.Project_Count));

      // Render Values on Top of Bars
      svg.selectAll(".value")
        .data(data)
        .enter()
        .append("text")
        .attr("class", "value")
        .attr("x", d => x(d.Category) + x.bandwidth() / 2)
        .attr("y", d => y(d.Project_Count) - 8)
        .attr("text-anchor", "middle")
        .text(d => d.Project_Count);

      // Axis Labels
      svg.append("text")
        .attr("class", "axis-label")
        .attr("x", width / 2)
        .attr("y", height - 20)
        .attr("text-anchor", "middle")
        .text("Technology Category");

      svg.append("text")
        .attr("class", "axis-label")
        .attr("transform", "rotate(-90)")
        .attr("x", -(height / 2))
        .attr("y", 25)
        .attr("text-anchor", "middle")
        .text("Number of Projects");
    }).catch(function(err) {
      console.log("Error loading CSV:", err);
    });
  </script>
</body>
</html>
```

OUTPUT



Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result:

A working interactive D3.js web bar chart was successfully created by loading external CSV data. The visualization accurately rendered dynamic SVG bars, numerical value labels, custom axes, and smooth hover color interactions across technological categories.